

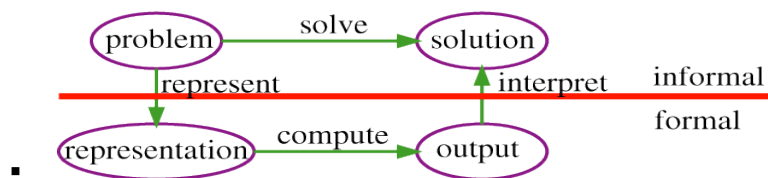
Representation and Mappings

1. Representation:

- **Definition:** Knowledge representation is the field of AI dedicated to representing information about the world in a form that a computer system can utilize to solve complex tasks.

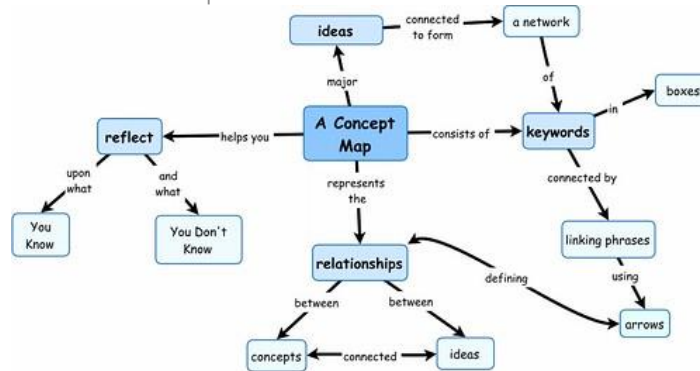
Artificial intelligence (AI) is based on the core idea of knowledge representation, which tries to capture and organize knowledge in a meaningful and structured fashion. It entails archiving data and making it available to AI systems so they may learn, reason, and make decisions based on knowledge.

- **Types:**
 - **Symbolic:** Uses symbols to represent knowledge (e.g., logic, frames).
 - **Sub-symbolic:** Uses patterns and connections (e.g., neural networks).

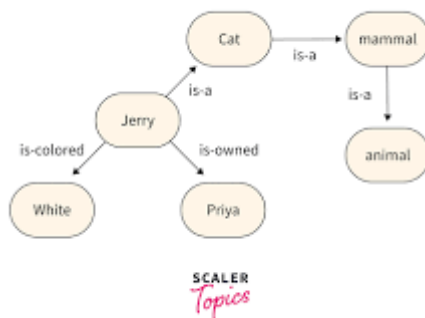


2. Mappings:

- **Mapping:** The process of associating one set of data with another. In AI, it involves translating real-world scenarios into a machine-understandable format.
- **Types of mappings:**
 - **Structural Mapping:** Aligning the structure of one domain with another.
 - **Semantic Mapping:** Aligning meanings across different representations.



Different Approaches



1. Logical Representation:

- **Predicate Logic:** Uses predicates and quantifiers to represent facts and rules.

Predicates are properties, additional information to better express the subject of the sentence. A quantified predicate is a proposition, that is, when you assign values to a predicate with variables it can be made a proposition.

Examples of Predicate Logic

In $P(x) : x > 5$, x is the subject or the variable and ' >5 ' is the predicate.

$P(7) : 7 > 5$ is a proposition where we are assigning values to the variable x , and it has a truth value, i.e. True.

- **Propositional Logic:** Uses simple statements that can be true or false.

A **proposition** is basically a declarative sentence that has a truth value. Truth value can either be true or false, but it needs to be assigned any of the two values and not be ambiguous. The purpose of using propositional logic is to analyze a statement, individually or compositely

Example of Propositional Logic

Let's consider some statements as follows:

1. If x is real, then $x^2 > 0$
2. What is your name?
3. $(a+b)^2 = 100$
4. This statement is false.
5. This statement is true.

These statements are not propositions because they do not have a truth value. They are ambiguous.

But the following statements:

1. $(a+b)^2 = a^2 + 2ab + b^2$
2. If x is real, then $x^2 \geq 0$
3. If x is real, then $x^2 < 0$
4. The sun rises in the east.
5. The sun rises in the west.

Are all propositions because they have a specific truth value, true or false.

The branch of logic that deals with proposition is propositional logic.

Propositional Logic	Predicate Logic
Propositional logic is the logic that deals with a collection of declarative statements which have a truth value, true or false.	Predicate logic is an expression consisting of variables with a specified domain. It consists of objects, relations and functions between the objects.

It is the basic and most widely used logic. Also known as Boolean logic. A proposition has a specific truth value, either true or false.	It is an extension of propositional logic covering predicates and quantification. A predicate's truth value depends on the variables' value.
Scope analysis is not done in propositional logic. Propositions are combined with Logical Operators or Logical Connectives like Negation($\neg$), Disjunction($\vee$), Conjunction($\wedge$), Exclusive OR($\oplus$), Implication($\Rightarrow$), Bi-Conditional or Double Implication($\Leftrightarrow$).	Predicate logic helps analyze the scope of the subject over the predicate. There are three quantifiers : Universal Quantifier ($\forall$) depicts for all, Existential Quantifier ($\exists$) depicting there exists some and Uniqueness Quantifier ($\exists!$) depicting exactly one. Predicate Logic adds by introducing quantifiers to the existing proposition.
It is a more generalized representation. It cannot deal with sets of entities.	It is a more specialized representation. It can deal with set of entities with the help of quantifiers.

2. Procedural Representation:

- **Scripts:** Predefined sequences of events (e.g., restaurant script).
- **Production Rules:** Condition-action pairs used in expert systems.

3. Network Representation:

- **Semantic Networks:** Graphs of concepts connected by relationships.
- **Frames:** Data structures that hold attributes and values for representing stereotyped situations.

4. Other Approaches:

- **Ontologies:** Formal representations of a set of concepts within a domain and the relationships between those concepts.
- **Bayesian Networks:** Probabilistic graphical models representing a set of variables and their conditional dependencies.

Issues in Knowledge Representation

1. **Expressiveness:** Balancing the richness of the representation with computational efficiency.

2. **Granularity:** The level of detail at which knowledge is represented.
3. **Incompleteness:** Dealing with incomplete or uncertain knowledge.
4. **Consistency:** Ensuring that the represented knowledge does not contain contradictions.
5. **Scalability:** Ability to handle large amounts of knowledge efficiently.
6. **Inference:** Mechanisms to derive new knowledge from existing knowledge.

Predicate Logic

Representation of Simple Facts in Logic

The most basic unit of knowledge representation in logic is a simple fact. These facts represent basic statements about the world that are true or false.

- **Logic Symbols: We use symbols to represent objects, relationships, and truth values.**
 - Constants: Represent specific objects (e.g., "Socrates").
 - Variables: Represent any object of a certain type (e.g., "x").
 - Predicates: Represent relationships between objects (e.g., "is_a(x, man)").

Example: The statement "Socrates is a man" can be represented in logic as: $\text{is_a}(\text{Socrates}, \text{man})$. Here, "Socrates" is a constant, "is_a" is a predicate indicating the "is-a" relationship, and "man" represents the category.

1. **Syntax:** Using variables, constants, functions, predicates, quantifiers, and connectives.
 - **Example:** $P(x)P(x)P(x)$ where PPP is a predicate and xxx is a variable.
2. **Semantics:** Interpretation of the syntax to give meaning.
 - **Example:** "All humans are mortal" can be represented as $\forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$ for all x, $(\text{Human}(x) \rightarrow \text{Mortal}(x)) \forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$.

Representing Instance and Isa Relationships:

Logic allows us to represent two important relationships between objects and categories:

- **Instance Relationship:** This shows that an object belongs to a specific category.
 - Example: "Tweety is a bird" can be written as: $\text{is_a}(\text{Tweety}, \text{bird})$. Here, "Tweety" is an instance of the category "bird".
 -
- **Isa Relationship ("is-a"):** This captures the hierarchical structure of knowledge by representing a subclass relationship between categories.
 - Example: "All birds are animals" can be expressed as: $\forall x (\text{bird}(x) \rightarrow \text{animal}(x))$. ("∀x" is the universal quantifier, meaning "for all x"). This statement says that anything that is a bird is also an animal.

1. **Instance Relationships:** Specific examples of a general category.

- **Example:** Human(Socrates)Human(Socrates)Human(Socrates) represents that Socrates is an instance of the category human.
- 2. **Isa Relationships:** Hierarchical relationships between categories.
 - **Example:** Isa(Dog,Animal)Isa(Dog, Animal)Isa(Dog,Animal) represents that dogs are a subclass of animals.

Computable Functions and Predicates

Predicate logic can represent not just simple relationships but also functions that operate on objects.

- **Functions:** These take one or more objects as input and produce a specific output object.
 - Example: "father(John)" could be a function that takes a person's name and returns their father's name.
 - **Computable Predicates:** Similar to functions, predicates can take objects as input and evaluate to true or false based on a defined rule.
 - Example: "greater_than(5, 3)" would be a predicate that evaluates to true because 5 is greater than 3.
1. **Functions:** Mappings from objects to objects.
 - **Example:** FatherOf(x)FatherOf(x)FatherOf(x) could return the father of xxx.
 2. **Predicates:** Functions that return true or false.
 - **Example:** GreaterThan(x,y)GreaterThan(x, y)GreaterThan(x,y) returns true if xxx is greater than yyy.

Resolution

Resolution is a powerful technique for automated reasoning in logic. It allows us to derive new logical statements (conclusions) from existing ones (premises). Here's the basic idea:

1. We convert both the premises and the desired conclusion into a specific format called clauses.
2. We apply a set of rules to identify complementary clauses (clauses with opposite literals).
3. By combining and resolving these complementary clauses, we eliminate some symbols and potentially derive a new clause.
4. If this new clause represents a tautology (always true statement), then the desired conclusion can be proven true based on the given premises.

Resolution is a complex topic, but understanding its core idea helps you grasp how logic can be used to reason about knowledge and draw conclusions automatically.

Resolution Rule: A single inference rule used in predicate logic and propositional logic for automated theorem proving.

Process: Convert all statements to a standard form (CNF), then iteratively apply the resolution rule to derive a contradiction.

Propositional Logic

Propositional logic is a simpler form of logic that deals with propositions, which are statements that can be true or false.

1. Representation:

- **Propositional Variables:** Represented by letters (e.g., P, Q, R). They stand for statements whose truth value we are interested in.
- **Logical Connectives:** These symbols combine propositions to form more complex statements.
 - **AND ($\wedge$):** Both propositions must be true (e.g., $P \wedge Q$).
 - **OR ($\vee$):** At least one proposition must be true (e.g., $P \vee Q$).
 - **NOT ($\neg$):** Negates a proposition (e.g., $\neg P$).
 - **Implies ($\rightarrow$):** If the first proposition (P) is true, then the second (Q) must also be true (e.g., $P \rightarrow Q$).
 - **Equivalent ($\leftrightarrow$):** Both propositions have the same truth value ($P \leftrightarrow Q$).
- **Example:** "It is raining today (P), OR I will go for a walk (Q)." This can be written as $P \vee Q$.

2. Inference:

Inference refers to the process of deriving new logical statements (conclusions) from existing ones (premises). Here are some common reasoning patterns:

- **Modus Ponens:** If P implies Q, and P is true, then Q is true. ($P \rightarrow Q, P \vdash Q$)
- **Modus Tollens:** If

Representation

1. Basic Elements:

- **Propositions:** Statements that are either true or false.
- **Connectives:** AND ($\wedge$), OR ($\vee$), NOT ($\neg$), IMPLIES ($\rightarrow$), IF AND ONLY IF ($\leftrightarrow$).

Inference

1. Methods:

- **Truth Tables:** Enumerate all possible truth values.
- **Natural Deduction:** Use inference rules (e.g., modus ponens, modus tollens).

Reasoning Patterns

1. Common Patterns:

- **Modus Ponens:** If $P \rightarrow Q$ and P is true, then Q is true.
- **Modus Tollens:** If $P \rightarrow Q$ and Q is false, then P is false.
- **Disjunctive Syllogism:** If $P \vee Q$ and $\neg P$, then Q .

Resolution

1. Propositional Resolution:

- Convert all formulas to Conjunctive Normal Form (CNF).
- Apply resolution rule iteratively to derive a contradiction.

First-order Logic

Representation

1. Syntax:

- **Variables:** $x, y, z, \dots$
- **Constants:** $a, b, c, \dots$
- **Functions:** $f(x), g(x, y), \dots$
- **Predicates:** $P(x), Q(x, y), \dots$
- **Quantifiers:** Universal ($\forall$), Existential ($\exists$)

Inference

1. Methods:

- **Unification:** Finding a substitution that makes different logical expressions identical.
- **Natural Deduction:** Using rules of inference to derive conclusions.
- **Resolution:** Extended to handle quantifiers.

Reasoning Patterns

1. Common Patterns:

- **Universal Instantiation:** From $\forall x P(x)$, infer $P(a)$ for any constant a .
- **Existential Instantiation:** From $\exists x P(x)$, infer $P(c)$ for some new constant c .

Resolution

1. First-order Resolution:

- **Process:**
 - Convert formulas to prenex normal form (all quantifiers at the front).
 - Skolemize the formula (eliminate existential quantifiers by introducing Skolem functions).

- Convert to CNF.
- Apply resolution rule iteratively.